

Supply Chain Shipment Tracking System

Complete Python Source Code, Configuration, Outputs and Rubric Mapping

This PDF contains the complete source code for the problem shown in the SIMATS Python Programming Lab screenshot. The project uses Python 3.x standard-library modules only.

50-Mark Rubric Coverage

Part	Implementation	Marks
(a)	Shipment base class + AirShipment, GroundShipment and SeaShipment subclasses with overridden delivery_days()	10
(b)	ConfigLoader reads rates.json and categories.csv	10
(c)	ShipmentManager calculations, CSV persistence and dynamic updates	12
(d)	Validation + custom exceptions + exception handling	8
(e)	Delivery report, receipts, activity log and delay alerts	10
(f)	Extensible category design and evaluation	10

main.py

```
from pathlib import Path

from config_loader import ConfigLoader
from exceptions import ShipmentError
from models import AirShipment, GroundShipment, SeaShipment, Location
from processor import ShipmentManager

BASE_DIR = Path(__file__).resolve().parent
CONFIG_DIR = BASE_DIR / "config"
DATA_DIR = BASE_DIR / "data"
OUTPUT_DIR = BASE_DIR / "output"

def display_shipment(shipment):
    print("\n--- Shipment Details ---")
    print("Tracking ID :", shipment.tracking_id)
    print("Category      :", shipment.category())
    print("Origin         :", shipment.origin)
    print("Destination    :", shipment.destination)
    print("Weight         :", shipment.weight, "kg")
    print("Status         :", shipment.status)
    print("ETA            :", shipment.delivery_days(), "days")

def create_sample_data(manager):
    """Create sample records only for a new/empty data file."""
    samples = [
        AirShipment(
            "AIR1001",
            Location("Chennai", "Tamil Nadu", "India"),
            Location("Delhi", "Delhi", "India"),
            15,
            5,
        ),
        GroundShipment(
            "GRD1002",
            Location("Bengaluru", "Karnataka", "India"),
            Location("Hyderabad", "Telangana", "India"),
            30,
            4,
        ),
        SeaShipment(
            "SEA1003",
            Location("Mumbai", "Maharashtra", "India"),
            Location("Kolkata", "West Bengal", "India"),
            120,
            10,
        ),
    ]
    for shipment in samples:
        manager.add_shipment(shipment)

def main():
    loader = ConfigLoader(
        CONFIG_DIR / "rates.json",
        CONFIG_DIR / "categories.csv",
    )
    settings = loader.load_settings()
    categories = loader.load_categories()
    print("=== SUPPLY CHAIN SHIPMENT TRACKING SYSTEM ===")
```

```

print("Loaded categories:", ", ".join(row["category"] for row in categories))

manager = ShipmentManager(DATA_DIR, OUTPUT_DIR, settings)
manager.load_shipments()

if not manager.shipments:
    create_sample_data(manager)

# Dynamic checkpoint/status updates.
try:
    manager.update_status(
        "AIR1001", "Chennai", "Tamil Nadu", "India",
        "In Transit", "Shipment departed origin."
    )
    manager.update_status(
        "GRD1002", "Hyderabad", "Telangana", "India",
        "Delivered", "Received by customer."
    )
    manager.update_status(
        "SEA1003", "Mumbai", "Maharashtra", "India",
        "Delayed", "Weather-related delay."
    )
except ShipmentError as error:
    print("Update error:", error)

# Invalid tracking ID demonstration.
try:
    manager.find_shipment("BAD")
except ShipmentError as error:
    print("Validation test:", error)

report = manager.generate_report()
receipts = manager.generate_receipts()

for shipment in manager.shipments.values():
    display_shipment(shipment)

print("\nGenerated output files:")
print(report)
print(receipts)
print(manager.log_file)
print(manager.alert_file)

if __name__ == "__main__":
    main()

```

models.py

```
from dataclasses import dataclass

@dataclass
class Location:
    city: str
    region: str
    country: str

    def __str__(self):
        return f"{self.city}, {self.region}, {self.country}"

@dataclass
class TrackingUpdate:
    tracking_id: str
    location: Location
    status: str
    timestamp: str
    remarks: str = ""

class Shipment:
    """Base class for all shipment categories."""

    def __init__(self, tracking_id, origin, destination, weight, expected_days):
        self.tracking_id = tracking_id
        self.origin = origin
        self.destination = destination
        self.weight = float(weight)
        self.expected_days = int(expected_days)
        self.status = "Created"
        self.updates = []

    def delivery_days(self):
        """Default delivery calculation. Subclasses override this."""
        return self.expected_days

    def update_status(self, update):
        self.status = update.status
        self.updates.append(update)

    def category(self):
        return self.__class__.__name__.replace("Shipment", "")

class AirShipment(Shipment):
    """Air shipments are faster than the normal estimate."""

    def delivery_days(self):
        return max(1, self.expected_days - 2)

class GroundShipment(Shipment):
    """Ground shipment uses the standard estimate."""

    def delivery_days(self):
        return self.expected_days

class SeaShipment(Shipment):
    """Sea shipments normally need additional time."""
```

```
def delivery_days(self):  
    return self.expected_days + 3
```

```
SHIPMENT_TYPES = {  
    "Air": AirShipment,  
    "Ground": GroundShipment,  
    "Sea": SeaShipment,  
}
```

config_loader.py

```
import csv
import json

class ConfigLoader:
    """Loads external JSON and CSV configuration files."""

    def __init__(self, json_path, csv_path):
        self.json_path = json_path
        self.csv_path = csv_path

    def load_settings(self):
        with self.json_path.open("r", encoding="utf-8") as file:
            return json.load(file)

    def load_categories(self):
        with self.csv_path.open("r", encoding="utf-8", newline="") as file:
            return list(csv.DictReader(file))
```

processor.py

```
import csv
from datetime import datetime
from pathlib import Path

from exceptions import (
    InvalidShipmentDataError,
    InvalidStatusError,
    InvalidTrackingIDError,
    ShipmentNotFoundError,
)
from models import Location, SHIPMENT_TYPES, TrackingUpdate

class ShipmentManager:
    """Core processing, persistence, reporting and logging logic."""

    def __init__(self, data_dir, output_dir, settings):
        self.data_dir = Path(data_dir)
        self.output_dir = Path(output_dir)
        self.settings = settings
        self.shipments = {}

        self.shipments_file = self.data_dir / "shipments.csv"
        self.updates_file = self.data_dir / "tracking_updates.csv"
        self.log_file = self.output_dir / "activity_log.txt"
        self.alert_file = self.output_dir / "alerts.txt"

        self.data_dir.mkdir(parents=True, exist_ok=True)
        self.output_dir.mkdir(parents=True, exist_ok=True)
        self._create_files_if_missing()

    def _create_files_if_missing(self):
        if not self.shipments_file.exists():
            with self.shipments_file.open("w", newline="", encoding="utf-8") as file:
                csv.writer(file).writerow([
                    "tracking_id", "category", "origin", "destination",
                    "weight", "expected_days", "status"
                ])
        if not self.updates_file.exists():
            with self.updates_file.open("w", newline="", encoding="utf-8") as file:
                csv.writer(file).writerow([
                    "tracking_id", "city", "region", "country",
                    "status", "timestamp", "remarks"
                ])
        self.log_file.touch(exist_ok=True)
        self.alert_file.touch(exist_ok=True)

    @staticmethod
    def validate_tracking_id(tracking_id):
        if not isinstance(tracking_id, str) or not tracking_id.strip():
            raise InvalidTrackingIDError("Tracking ID cannot be empty.")
        tracking_id = tracking_id.strip().upper()
        if len(tracking_id) < 4 or not tracking_id.replace("-", "").isalnum():
            raise InvalidTrackingIDError(
                "Tracking ID must have at least 4 letters/numbers."
            )
        return tracking_id

    @staticmethod
    def parse_location(text):
        parts = [part.strip() for part in text.split(",")]
        if len(parts) != 3 or any(not part for part in parts):
```

```

        raise InvalidShipmentDataError(
            "Location must be in City, Region, Country format."
        )
    return Location(*parts)

def add_shipment(self, shipment):
    shipment.tracking_id = self.validate_tracking_id(shipment.tracking_id)
    if shipment.tracking_id in self.shipments:
        raise InvalidShipmentDataError("Tracking ID already exists.")
    if shipment.weight <= 0:
        raise InvalidShipmentDataError("Weight must be greater than zero.")
    if shipment.expected_days <= 0:
        raise InvalidShipmentDataError("Expected days must be greater than zero.")
    if shipment.category() not in SHIPMENT_TYPES:
        raise InvalidShipmentDataError("Unsupported shipment category.")

    self.shipments[shipment.tracking_id] = shipment
    with self.shipments_file.open("a", newline="", encoding="utf-8") as file:
        csv.writer(file).writerow([
            shipment.tracking_id,
            shipment.category(),
            shipment.origin,
            shipment.destination,
            shipment.weight,
            shipment.expected_days,
            shipment.status,
        ])
    self.log(f"Created shipment {shipment.tracking_id} ({shipment.category()}).")

def load_shipments(self):
    with self.shipments_file.open("r", encoding="utf-8", newline="") as file:
        for row in csv.DictReader(file):
            category = row["category"].strip()
            cls = SHIPMENT_TYPES.get(category)
            if cls is None:
                continue
            shipment = cls(
                row["tracking_id"],
                self.parse_location(row["origin"]),
                self.parse_location(row["destination"]),
                row["weight"],
                row["expected_days"],
            )
            shipment.status = row["status"]
            self.shipments[shipment.tracking_id] = shipment

    if self.updates_file.exists():
        with self.updates_file.open("r", encoding="utf-8", newline="") as file:
            for row in csv.DictReader(file):
                shipment = self.shipments.get(row["tracking_id"])
                if shipment:
                    update = TrackingUpdate(
                        row["tracking_id"],
                        Location(row["city"], row["region"], row["country"]),
                        row["status"],
                        row["timestamp"],
                        row["remarks"],
                    )
                    shipment.update_status(update)

def update_status(self, tracking_id, city, region, country, status, remarks=""):
    tracking_id = self.validate_tracking_id(tracking_id)
    if tracking_id not in self.shipments:
        raise ShipmentNotFoundError(f"No shipment found for {tracking_id}.")
    allowed = self.settings.get("statuses", [])

```



```

        if status not in allowed:
            raise InvalidStatusError(
                "Invalid status. Allowed: " + ", ".join(allowed)
            )

        shipment = self.shipments[tracking_id]
        if shipment.status == status and remarks:
            return False # Prevent repeated identical demo updates on every run.

        location = Location(city.strip(), region.strip(), country.strip())
        if not all([location.city, location.region, location.country]):
            raise InvalidShipmentDataError("Checkpoint location cannot be empty.")

        update = TrackingUpdate(
            tracking_id,
            location,
            status,
            datetime.now().strftime("%Y-%m-%d %H:%M:%S"),
            remarks,
        )
        shipment.update_status(update)

        with self.updates_file.open("a", newline="", encoding="utf-8") as file:
            csv.writer(file).writerow([
                tracking_id, location.city, location.region, location.country,
                status, update.timestamp, remarks
            ])

        self.log(f"{tracking_id}: status changed to {status} at {location}.")
        if status == "Delayed":
            with self.alert_file.open("a", encoding="utf-8") as file:
                file.write(
                    f"ALERT | {update.timestamp} | {tracking_id} | "
                    f"Delayed at {location} | {remarks}\n"
                )
        return True

def find_shipment(self, tracking_id):
    tracking_id = self.validate_tracking_id(tracking_id)
    if tracking_id not in self.shipments:
        raise ShipmentNotFoundError(f"No shipment found for {tracking_id}.")
    return self.shipments[tracking_id]

def generate_report(self):
    report_file = self.output_dir / "delivery_report.csv"
    with report_file.open("w", newline="", encoding="utf-8") as file:
        writer = csv.writer(file)
        writer.writerow([
            "tracking_id", "category", "origin", "destination", "weight_kg",
            "status", "expected_days", "calculated_delivery_days", "delay_analysis"
        ])
    for shipment in self.shipments.values():
        delay = "Delayed" if shipment.status == "Delayed" else "On Time / Normal"
        writer.writerow([
            shipment.tracking_id,
            shipment.category(),
            shipment.origin,
            shipment.destination,
            shipment.weight,
            shipment.status,
            shipment.expected_days,
            shipment.delivery_days(),
            delay,
        ])
    return report_file

```

```

def generate_receipts(self):
    receipt_file = self.output_dir / "receipts.txt"
    with receipt_file.open("w", encoding="utf-8") as file:
        for shipment in self.shipments.values():
            file.write("=" * 60 + "\n")
            file.write("SUPPLY CHAIN SHIPMENT RECEIPT\n")
            file.write("=" * 60 + "\n")
            file.write(f"Tracking ID : {shipment.tracking_id}\n")
            file.write(f"Category      : {shipment.category()}\n")
            file.write(f"Origin       : {shipment.origin}\n")
            file.write(f"Destination  : {shipment.destination}\n")
            file.write(f"Weight       : {shipment.weight} kg\n")
            file.write(f>Status       : {shipment.status}\n")
            file.write(f"ETA          : {shipment.delivery_days()} days\n")
            file.write("=" * 60 + "\n\n")
    return receipt_file

def log(self, message):
    with self.log_file.open("a", encoding="utf-8") as file:
        file.write(
            f"{datetime.now().strftime('%Y-%m-%d %H:%M:%S')} | {message}\n"
        )

```

exceptions.py

```
class ShipmentError(Exception):
    """Base exception for shipment tracking errors."""

class InvalidTrackingIDError(ShipmentError):
    """Raised when a tracking ID is invalid."""

class ShipmentNotFoundError(ShipmentError):
    """Raised when a shipment is not found."""

class InvalidStatusError(ShipmentError):
    """Raised when a status is not allowed."""

class InvalidShipmentDataError(ShipmentError):
    """Raised when shipment data is invalid."""
```

README.txt

SUPPLY CHAIN SHIPMENT TRACKING SYSTEM
=====

Python 3.x standard library only.

HOW TO RUN

1. Extract the ZIP file.
2. Open the project folder in VS Code, IDLE or PyCharm.
3. Open a terminal in the project folder.
4. Run:

```
python main.py
```

FILES

```
main.py           : Main program and demonstration.
models.py         : Location, TrackingUpdate, Shipment and subclasses.
config_loader.py  : Reads JSON and CSV configuration files.
processor.py      : Core processing, persistence, reports and logs.
exceptions.py     : Custom exception classes.
config/rates.json : External settings and allowed statuses.
config/categories.csv: External shipment category configuration.
data/shipments.csv : Persistent shipment details.
data/tracking_updates.csv : Persistent checkpoint/status updates.
output/delivery_report.csv : Delivery and delay analysis report.
output/receipts.txt : Shipment receipts.
output/activity_log.txt : Shipment activity log.
output/alerts.txt  : Delay alerts.
```

RUBRIC COVERAGE

- (a) 10 marks: Shipment base class plus AirShipment, GroundShipment and SeaShipment subclasses. Each category overrides delivery_days().
- (b) 10 marks: ConfigLoader reads rates.json and categories.csv externally.
- (c) 12 marks: ShipmentManager performs calculations, CSV read/write, checkpoint updates and persistent storage.
- (d) 8 marks: Input validation, exception handling and custom error types InvalidTrackingIDError, ShipmentNotFoundError, InvalidStatusError and InvalidShipmentDataError.
- (e) 10 marks: delivery_report.csv, receipts.txt, activity_log.txt and alerts.txt are generated.
- (f) 10 marks: Extensible OOP design. A new shipment category can be added by creating a subclass and registering it in SHIPMENT_TYPES without changing the core report/update algorithms.

SAMPLE TEST

The program tests an invalid tracking ID (BAD) and demonstrates valid status updates for air, ground and sea shipments.

config/rates.json

```
{
  "company": "Supply Chain Logistics",
  "statuses": [
    "Created",
    "In Transit",
    "At Checkpoint",
    "Delivered",
    "Delayed"
  ],
  "max_weight_kg": 1000,
  "category_notes": {
    "Air": "Fast delivery; subclass reduces normal ETA by 2 days.",
    "Ground": "Standard road delivery; normal ETA is used.",
    "Sea": "Sea freight; subclass adds 3 days to normal ETA."
  }
}
```

config/categories.csv

```
category,description
Air,Fast air shipment
Ground,Standard road shipment
Sea,Sea freight shipment
```

data/shipments.csv

```
tracking_id,category,origin,destination,weight,expected_days,status
AIR1001,Air,"Chennai, Tamil Nadu, India","Delhi, Delhi, India",15.0,5,Created
GRD1002,Ground,"Bengaluru, Karnataka, India","Hyderabad, Telangana, India",30.0,4,Created
SEA1003,Sea,"Mumbai, Maharashtra, India","Kolkata, West Bengal, India",120.0,10,Created
```

data/tracking_updates.csv

```
tracking_id,city,region,country,status,timestamp,remarks
AIR1001,Chennai,Tamil Nadu,India,In Transit,2026-08-25 05:51:38,Shipment departed origin.
GRD1002,Hyderabad,Telangana,India,Delivered,2026-08-25 05:51:38,Received by customer.
SEA1003,Mumbai,Maharashtra,India,Delayed,2026-08-25 05:51:38,Weather-related delay.
```


output/delivery_report.csv

```
tracking_id,category,origin,destination,weight_kg,status,expected_days,calculated_delivery_days,delay_analysis
AIR1001,Air,"Chennai, Tamil Nadu, India","Delhi, Delhi, India",15.0,In Transit,5,3,On Time / Normal
GRD1002,Ground,"Bengaluru, Karnataka, India","Hyderabad, Telangana, India",30.0,Delivered,4,4,On Time / Normal
SEA1003,Sea,"Mumbai, Maharashtra, India","Kolkata, West Bengal, India",120.0,Delayed,10,13,Delayed
```

output/receipts.txt

```
=====
SUPPLY CHAIN SHIPMENT RECEIPT
=====
Tracking ID : AIR1001
Category    : Air
Origin      : Chennai, Tamil Nadu, India
Destination : Delhi, Delhi, India
Weight      : 15.0 kg
Status      : In Transit
ETA         : 3 days
=====

=====
SUPPLY CHAIN SHIPMENT RECEIPT
=====
Tracking ID : GRD1002
Category    : Ground
Origin      : Bengaluru, Karnataka, India
Destination : Hyderabad, Telangana, India
Weight      : 30.0 kg
Status      : Delivered
ETA         : 4 days
=====

=====
SUPPLY CHAIN SHIPMENT RECEIPT
=====
Tracking ID : SEA1003
Category    : Sea
Origin      : Mumbai, Maharashtra, India
Destination : Kolkata, West Bengal, India
Weight      : 120.0 kg
Status      : Delayed
ETA         : 13 days
=====
```

output/activity_log.txt

```
2026-08-25 05:51:38 | Created shipment AIR1001 (Air).
2026-08-25 05:51:38 | Created shipment GRD1002 (Ground).
2026-08-25 05:51:38 | Created shipment SEA1003 (Sea).
2026-08-25 05:51:38 | AIR1001: status changed to In Transit at Chennai, Tamil Nadu, India.
2026-08-25 05:51:38 | GRD1002: status changed to Delivered at Hyderabad, Telangana, India.
2026-08-25 05:51:38 | SEA1003: status changed to Delayed at Mumbai, Maharashtra, India.
```

output/alerts.txt

ALERT | 2026-08-25 05:51:38 | SEA1003 | Delayed at Mumbai, Maharashtra, India | Weather-related delay.